



第六章 AutoLISP语言

6.1 Visual LISP集成开发环境

6.2 AutoLISP语言基础

6.3 基本函数

6.4 程序设计简介

6.5 应用举例





一. 概述

1. AutoCAD 2018是一个通用CAD平台，它不仅具有强大的绘图、编辑功能，而且具有开放的体系结构，允许用户对其进行二次开发，以满足绘图过程中的特殊要求。
2. AutoCAD 允许用AutoLISP、Visual Basic、VBA、Visual C++等多种高级语言进行二次开发。
3. 本课主要介绍如何用AutoLISP进行AutoCAD 的二次开发。





一. LISP、AutoLISP及Visual LISP的关系

- LISP语言

- LISP语言是一种表处理解释型高级语言，诞生于20世纪50年代末期，是人工智能学科领域广泛应用的程序设计语言（不只是应用在AutoCAD上）。

- AutoLISP

- AutoLISP是一种专门针对AutoCAD进行二次开发的高级编程语言，它嵌套于AutoCAD内部，是LISP语言与AutoCAD有机结合的产物。





① AutoLISP是AutoCAD二次开发的最主要的工具

- 具有强大的图形处理功能
- 用户可以利用AutoLISP语言实现对AutoCAD当前图形数据库的直接访问和修改
- 自定义AutoCAD命令
- 开发参数化绘图程序

② AutoLISP语言是函数式语言

- AutoLISP程序由一个一个函数的组成，用户编写AutoLISP程序，实际上就是调用若干个函数。
- 用户可用任一种ASCII文本编辑软件来编辑AutoLISP源程序，AutoLISP源程序的后缀名为 .lsp 。





- Visual LISP（简称VLISP）
 - VisualLISP是用AutoLISP语言进行AutoCAD二次开发的一个集成开发环境(Integrated Development Environment, IDE)。
 - 它提供有众多工具，可使用户方便地用AutoLISP语言进行AutoCAD的二次开发。如编写程序代码、调试程序等等。





6.1 Visual LISP集成开发环境简介

一、启动

1. 命令行启动: **VLISP**
2. 菜单栏启动: “工具” → “**AutoLISP**”
→ “**Visual LISP编辑器**”

说明

- 只能在AutoCAD环境中启动Visual LISP。





二、界面组成（6部分）

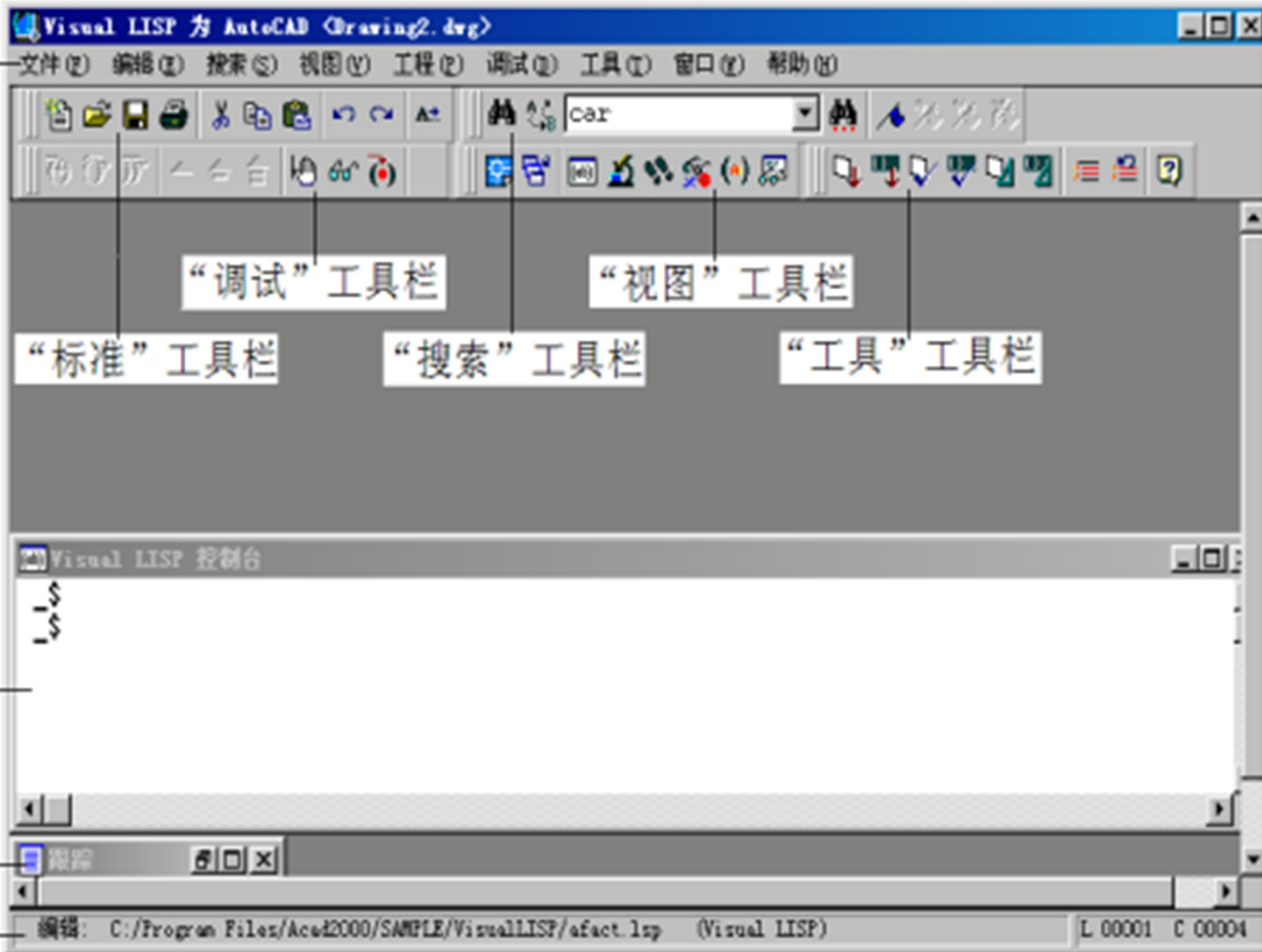
1. 菜单栏
2. 工具栏
3. 状态栏
4. **LISP**源程序编辑窗口（要新建）
5. 控制台窗口
6. 搜索窗口（跟踪窗口）

VLISP集成开发环境的主界面如下图:





菜单栏



控制台窗口

搜索窗口

状态栏

编辑: C:/Program Files/Acad2000/SAMPLE/VisualLISP/afact.lsp (Visual LISP)

L 00001 C 00004



- 工具栏（基本等价于菜单栏）
- 5个工具栏：“标准工具栏”、“搜索工具栏”、“调试工具栏”、“视图工具栏”和“工具工具栏”。
- 显示工具栏：[视图]菜单—[工具栏]
- **LISP源程序编辑窗口**
- 用来编写、调试AutoLISP程序，该编辑器具有以下特点：
 - 程序代码以不同的颜色显示
 - 具有自动缩进功能





● 控制台窗口

- 利用控制台窗口可以直接输入并运行 **AutoLISP** 的命令，以观看运行结果。
- 控制台窗口功能归纳：
 - 输入 **表**，显示表的返回值
 - 例如：在控制台中输入
 - **(setq x1 10)** 回车 **(setq x2 20)** 回车
 - 查看变量的值
 - 例如：
 - **x1** 回车 **x2** 回车
 - 回显输入情况
 - 按下 **Tab** 键，可在控制台窗口中显示出以前输入的内容（**Shift + Tab** 方向相反）





- 状态栏
 - Visual LISP在状态栏上显示出与当前操作所对应的信息。
 - 例如，当把光标放到某一菜单项或某一工具栏按钮上时，在状态栏上会显示出该菜单项或按钮的功能说明。
- 跟踪窗口
 - 显示程序调试时的错误信息。





三、二次开发的基本过程

- 编写 LISP 源程序
 - 可以用任何文本编辑器编写LISP源程序（如：Visual LISP、记事本、Word、C++编辑器等）。只要将文件存储为：**.lsp** 即可。
- 在 **AutoCAD 环境**中运行 LISP 源程序中定义好的**命令**。
 - **注意：自定义的命令不能与 AutoCAD 内部命令同名**
 - **LISP编写的所有绘图程序在运行时都必须得关掉对象捕捉**

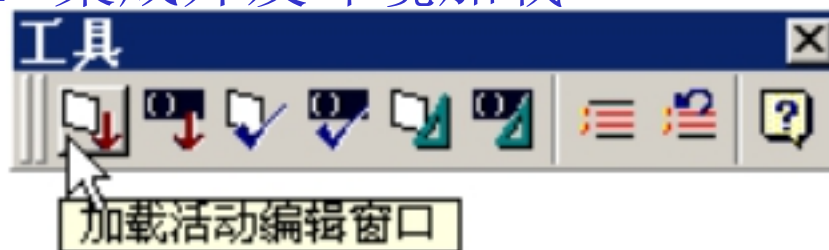




- 加载、运行AutoLISP 源程序（两种方式）

- 通过Visual LISP 集成开发环境加载

- 工具栏加载



- 菜单加载

- [文件] → [加载文件...]

- [工具] → [加载编辑器中的文字]

- 运行

- “defun c:<函数名>”在控制台窗口中，在\$提示符后输入： (c:函数名) 后回车

- “defun <函数名>”在命令窗口中的“在控制台窗口中，在\$提示符后输入： (函数名)，如有参数在()内加上相应参数值，函数名与参数值之间空一格，回车





例如：

```
(defun comp (a b / c)
  (setq c (getint "\n Input a Integer "))
  (if (<= (+ a b) c)
    "(a+b) LESS OR EQUAL c"
    "(a+b) MORE THAN c")
  )
```

- 加载后，在控制台窗口中，在\$提示符后输入：
(comp 1 2) 后回车，执行

Input a Integer 10

"(a+b) LESS OR EQUAL c"





例如：

```
(defun c:score ()  
  (setq c (getint "\n Input a Integer "))  
  (cond ((<= c 60) "No Pass")  
        ((<= c 85) "Pass")  
        ((> c 85) "Excellent")  
  )  
)
```

- 加载后，在控制台窗口中，在\$提示符后输入：
- (c:score)后回车，执行

Input a Integer 10
"No Pass"





① 在 AutoCAD 环境中加载、运行AutoLISP程序

- 命令行加载
 - **APpload**
- 菜单加载
 - **[工具] → [AutoLISP] → [加载...]**
- 运行
 - ✓ “defun c:<函数名>”在命令窗口中的“命令：”提示行输入：函数名，回车
 - ✓ “defun <函数名>”在命令窗口中的“命令：”提示行输入：(函数名)，如有参数在（）内加上相应参数值，函数名与参数值之间空一格，回车





例如：

```
(defun c:score ()  
  (setq c (getint "\n Input a Integer "))  
  (cond ((<= c 60) "No Pass")  
        ((<= c 85) "Pass")  
        ((> c 85) "Excellent")  
  )  
)
```

加载后，在命令窗口中的命令行键入：**score** 回车，
执行：

Input a Integer 10
"No Pass"





例如：

```
(defun score ()  
  (setq c (getint "\n Input a Integer "))  
  (cond ((<= c 60) "No Pass")  
        ((<= c 85) "Pass")  
        ((> c 85) "Excellent")  
  )  
)
```

一. 加载后，在命令窗口中的命令行键入：(score)后回车，执行：

Input a Integer 10

"(a+b) LESS OR EQUAL c"





例如：

```
(defun comp (a b / c)
  (setq c (getint "\n Input a Integer "))
  (if (<= (+ a b) c)
    "(a+b) LESS OR EQUAL c"
    "(a+b) MORE THAN c")
)
```

- 加载后，在命令窗口中的命令行键入：(comp 1 2)
后回车，执行

Input a Integer 10

"(a+b) LESS OR EQUAL c"





6.2 AutoLISP语言基础

■ AutoLISP语言的特点

- AutoLISP语言在普通Lisp语言的基础上，扩充了许多适用于AutoCAD应用的特殊功能，并且可直接调用几乎全部的AutoCAD命令。它是一种解释性程序设计语言。
- 例如： `(command "line" x1 x2 "")`
- 调用AutoCAD中的绘直线的命令





- AutoLISP语言是函数式语言
- AutoLISP程序就是由函数按一定的语法结构组合而成，以完成用户特定的需要。
函数包括：内部函数和用户自定义函数。
 - 例如：(setq x1 (getpoint))
 - 赋值函数中调用了取点坐标函数
- AutoLISP程序的结构是一种表结构
 - 表与元素
 - 一对“ () ” 称为一个表
 - 例如：(setq x1 10)
 - setq、x1、10称为表中的元素





① 表的格式

- (函数名 [<参数1>] [<参数1 >]..... [<参数n >])
- 说明
 - 表的书写采用“前缀表示法”，“函数名”必须放在表中的第一个元素处；其他元素是函数的参数
 - 函数名和各参数之间用空格分隔
 - 例如，表达式 $X = (A+B)*C$ 在“表”中书写为：
 - (setq X (* (+ A B) C)) 的形式。

② 表可以嵌套调用（“括号”必须左右匹配）

- 例如：(setq x1 (getpoint))
- 有两个表，大表中嵌套有小表。setq、x1、(getpoint)为大表中的元素；getpoint为小表中的元素。





① 表的执行过程

- “表”的执行过程是由内向外执行，即从最内层表开始逐步到最外层表，每个“表”都会返回一个值，该值可被外层表引用。
 - 例如， $X = (A+B)*C$
 - `(setq X (* (+ A B) C))` 的形式。
- AutoLISP程序中的“注释”
 - ① 注释以分号“`;`”开头，
 - ② 它可以单独占一行，也可以位于某一行的后面。

学习AutoLISP语言的核心就是要掌握AutoLISP的“函数”和“表结构”；熟练使用AutoCAD命令。





■ AutoLISP语言基础

1、Setq函数

1) 格式

(setq sym1 expr1 [sym2 expr2] ...)

关键字

符号1

表达式1

符号2及其表达式（可选）

还可以有多个
符号、表达式





1、Setq函数

1) 格式

2) 作用

3) 举例

格式: (setq sym1 expr1 [sym2 expr2] ...)

作用: 把expr1赋值给sym1, 把expr2赋值给sym2, ..., 可同时为多个变量赋值, 但要求成对出现, 只返回最后一个变量的值, 见例2.

例 1:

```
(setq a 2)
(setq b 2.5)
(setq c "mystring")
(setq ts a)
(setq ts b)
(setq ts c)
```

例 2:

```
(setq a 3 b 4.5 c "mystring")
(setq ts a)
(setq ts b)
(setq ts c)
```





1、Setq函数

2、数据类型

1) 简介

AutoLISP有十种基本的数据类型：整数、实数、字符串、符号、表、用户定义函数、文件描述符、实体名、选择集、内部函数、函数分页表等。

其中最最基本的数据类型是整数、实数、字符串；符号、表、用户定义函数。





1、Setq函数

2、数据类型

1) 简介

2) 整/实数和字符串

整型数：是由0, 1, 2, ..., 9, +或- 这12个字符组成的整数。例如：+230、450、-18等均为整型数。

AutoLISP的整型数是32位，其取值范围为-2147483648~+2147483647。

实型数：就是实数（有小数点）
实型数既可以用十进制表示，也可以用科学计数法表示。例如：13.2、0.045、4.5e-2

字符串：是用双引号(“”)括起来的字符序列
AutoLISP对字符串有如下约定：
字符串中，字母有大小写之分，空格也是一个有意义的字符。
字符串的长度指位于双引号之间的字符个数。如果在字符串的定界符之间无任何字符，称其为空串，它的长度为零。



1、Setq函数

2、数据类型

1) 简介

2) 整/实数和字符串

I. 特殊字符（控制字符）

控制字符	功 能	用ASCII码表示
\\	输出 \	\\114
\"	输出 "	\042
\e	Esc	\033
\n	换行	\012
\r	回车	\015
\t	TAB(制表位)	\011

表中的 e、n、r、t 必须为小写字母。





1、Setq函数

2、数据类型

1) 简介

2) 整/实数和字符串

3) 符号

符号：用来存放数据,也称变量。
可用作变量或函数名。

说明：

1.在AutoLISP语言中，符号可以包含除下列字符以外的任何可以打印的字符：

- (左括号；
-) 右括号；
- . 点符号；
- ' 单引号；
- " 双引号；
- ； 分号。

2.在AutoLISP中，符号的大小写等效，但第一个字符不允许以数字开始；符号的长度不限，单一般在6个字符以内，否则，符号名太长会使程序可读性差，不易记忆。

3.有三个特殊的符号，即PI,T及NIL。



1、Setq函数

2、数据类型

1) 简介

2) 整/实数和字符串

3) 符号

4.不需要说明类型，由它的值的类型所决定；

5.类型可以任意改变，值不同类型也随之改变；

6.一个符号可以是全局变量，也可以是局部变量。若要看某个符号的当前约束值，可在命令状态下，键入一个感叹号“！”，后紧跟着要查看的符号即可；

7.用户在编写程序时，程序中所定义的符号名不能与系统中的函数和符号名相同，否则，后面的定义将取代已有的定义，从而引起难以检查的错误。



1、Setq函数

2、数据类型

1) 简介

2) 整实数和字符串

3) 符号

举例 1:

```
(setq a 123 r 3.45 s "Hello!" x '(a b c))
```

输入:

```
(type 'a)
```

```
(type a)
```

```
(type r)
```

```
(type s)
```

```
(type x)
```

返回:

```
SYM
```

```
INT
```

```
REAL
```

```
STR
```

```
LIST
```





1、Setq函数

2、数据类型

1) 简介

2) 整实数和字符串

3) 符号

4) 表

表：用圆括号括起来的单元称为表，是构成符号表达式的基本单元。

作用如同高级语言中的数组。

表中的项称为表的元素。

表由元素组成，元素之间用空格分开，放入括号中就构成了表。

```
(setq begin "Begin AutoLisp:")
```

↑ ↑ ↑
元素：是数据的基本单位，元素可以是各种类型的数据。





1、Setq函数

2、数据类型

1) 简介

2) 整实数和字符串

3) 符号

4) 表

举例 2:

```
(list 100 200)
```

List函数可将任意数目的表达式串在一起组成一个表;

返回 (100 200) ;

```
(setq pt (list 100 200))
```

获得点pt的坐标;

```
(type pt)
```

返回 “LIST”，说明是表





1、Setq函数

2、数据类型

3、命令函数

1) 格式

(Command para1 [para2] ...)

函 数
关键字

参数1：命令关键字
——必须是字符串

参数2（可选）

还可以
有多个
参数，
由命令
类型定





1、Setq函数

2、数据类型

3、命令函数

1) 格式

2) 作用

3) 举例

举例 1:

```
(command "line" "100,120" "23,116" "")
```

或

```
(command "line" '(100 120) '(23 116) "")
```

“空格” 代表回车;

最后的一对双引号 “” 代表一次回车,

使重新出现 “命令: ” 提示符,

以便继续送入Autocad命令。

格式: (Command para1 [para2] ...)

作用: 执行AutoCAD 命令

举例 2 (可整体):

```
(setq p1 (list 50 0))
```

```
(setq p2 (list 100 100))
```

```
(setq p3 (list 100 200))
```

```
(command "line" p1 p2 p3 "")
```

说明: 前一例是CAD中点的键盘输入形式, 这里是表结构表示。

错误:

```
(command "line" (0,0) (100,100) "")
```

```
(command line p1 p2 p3 "")
```



1、Setq函数

2、数据类型

3、命令函数

1) 格式

2) 作用

3) 举例

格式: (Command para1 [para2] ...)

作用: 执行AutoCAD 命令

举例 3 (可整体) :

```
(setq point (list 50 50) tt "AutoLISP")
```

```
(command "text" point 25 0 tt)
```

说明: 这里“text”字符串是命令关键字, point是对齐点, 25是文本高度, 0是旋转角度, tt是要书写的文本。

参数的顺序必须与命令要求一致。





1、Setq函数

2、数据类型

3、命令函数

4、polar函数

作用：以某点为基点，计算一个点（的坐标）。

(polar <点> <角度> <距离>)

函数关键字

基点：

要计算的点与基点的极坐标夹角

要计算的点与基点之间的距离





1、Setq函数

2、数据类型

3、命令函数

4、polar函数

格式：(polar <点> <角度> <距离>)

作用：以某点为基点，计算一个点（的坐标）。

举例 1（可整体）：

```
(setq p1 (list 0 0))
```

```
(setq p2 (polar p1 0 100))
```

```
(setq p3 (polar p2 (/ pi 2) 100))回车
```

```
(0 0)
```

```
(100.0 0.0)
```

```
(100.0 100.0)
```



1、Setq函数

2、数据类型

3、命令函数

4、polar函数

5、其他函数

1) setvar

作用：为AutoCAD的内存变量
设置新值。

(setvar <变量名> <值>)

函 数关键字

CAD的内存变量

要设置的新值

举例：

(setvar "DIMTXT" 0.18)

指定标注文字的高度





1、Setq函数

2、数据类型

3、命令函数

4、polar函数

5、其他函数

1) setvar

2) getvar

作用：读取AutoCAD的内存变量的值。

(getvar <变量名>)

函 数关键字

AutoCAD的内存变量

举例：

(setq a (getvar "DIMTXT"))

读取标注文字的高度



6.3 基本函数

1、数学函数

AutoLISP中没有运算符的概念，所有运算符都是以函数的形式定义的。AutoLISP提供了丰富的函数来处理数据。

下面先介绍数学函数。

语法格式	说明(四则运算)
(+ <Numb1>[Numb2]...)	求和函数：返回各数的和
(- <Numb1>[Numb2]...)	减法函数：返回Numb1减其它数
(* <Numb1>[Numb2]...)	求积函数：返回各数的积
(/ <Numb1>[Numb2]...)	除法函数：Numb1/Numb2/Numb3/



1、数学函数

举例 1(可整体)：

```
(setq h0 15)
(setq df 10)
(setq x0 (* 1.6 h0 df))
(setq y0 (* 1 h0 df))
15
10
240.0
150
```

说明：注意x0是实数；
y0是整数。

举例 2 (可整体)：

```
(setq x0 (/ pi 2))
(setq x1 (+ 1 2))
(setq x1 (- 1 2))
(setq y0 (* (/ 3 2) pi))
(setq y1 (* (/ 3.0 2) pi))
1.5708
3
-1
3.14159
4.71239
```

说明：注意pi是常数；
y0与y1区别。



1、数学函数

语法格式	说明 (初等函数)
(max<Numb1>[Numb2]...)	返回值为Number中的最大值。
(min<Numb1>[Numb2]...)	返回值为Number中的最小值。
(rem<Numb1>[Numb2]...)	返回Numbe1/Numbe2/... 的余数。
(sqrt <Number>)	平方根函数。
(abs <Number>)	返回Number的绝对值。
(1+ <Number>)	返回Number加1的值，1与+之间不能有空格。
(1- <Number>)	返回Number减1的值，1与-之间不能有空格。





1、数学函数

举例 1(可整体)：

(max 1 2 3)

(min 1 2 3)

(rem 3 2)

(sqrt 100)

(abs -1)

(1+ 3)

(+ 1 3)

3

1

1

10.0

1

4

4



1、数学函数

语法格式	说明 (科学计算)
(expt<Numb1><Numb2>)	返回<Numb1>的<Numb2>次幂。
(exp <Number>)	返回EXP (Number)， e的Number次幂
(log<Number>)	自然对数函数ln (Number)。
(sin <Number>)	返回sin (Number)。
(cos (Number>)	返回cos (Number)。
(atan<N1>[N2])	返回arctg (N1) 或arctg (N1/N2)。





1、数学函数

举例 1(可整体)：

(expt 3 2)

(exp 1)

(log (exp 1))

(sin (/ pi 6))

(cos (/ pi 3))

(atan 1)

9

2.71828

1.0

0.5

0.5

0.785398



1、 数学函数

2、 交互函数

交互函数能实现人机对话, 当程序运行遇到此类函数时, 要求输入相应数据, 由此来决定程序的流向和运行。

语法格式	说明
Print <expression>	输出表达式
Princ <expression>	输出表达式（处理控制符）

控制字符	字符含义
\nnn	八进制码为 nnn 的字符
\e	ESC 字符
\n	回车换行
\r	回车（不换行）
\t	TAB 字符（移到下一制表位）





1、 数学函数

2、 交互函数

举例(可整体)：

```
(setq string " \n asd \n bsc \t bsc \n")
```

```
" \n asd \n bsc \t bsc \n"
```

```
(print string)
```

```
" \n asd \n bsc \t bsc \n"
```

```
(princ string)
```

```
asd
```

```
bsc    bsc
```

说明： 1) 注意两函数的差别；
2) 返回值



1、数学函数

2、交互函数

语法格式	说明 (数串点输入)
<code>(getint[prompt])</code> <code>(getreal[prompt])</code>	从命令窗口根据提示输入一个整数和实数。 <code>prompt</code> 是提示信息字符串， <code>prompt</code> 若为空， 则无提示信息。
<code>(getstring [T]</code> <code>[prompt])</code>	从命令窗口根据提示输入一个字符串， 选项[T]若不为 <code>nil</code> ，则输入字符串中 可以包含空格，否则遇空格结束。
<code>(getpoint [base]</code> <code>[prompt])</code>	从命令窗口根据提示输入一个点的坐标(如： <u>0,0,0</u>)，或从屏幕上拾取一点，选项 <code>base</code> 定 一个基点，执行时系统将从该基点引出一条 橡皮线。函数将返回该点坐标。



1、数学函数

2、交互函数

举例（必须单句执行, 等候输入）：

```
(getint "Input a integer: ")
```

```
(getreal "Input a real: ")
```

```
(getstring "Input a no space string:")
```

； 遇到空格结束

```
"NAME"
```

```
(getstring T " Input a string:")
```

； 输入的字符串中可有空格

```
"MY NAME IS"
```

```
(setq p1 (getpoint "input first point:")) 或
```

```
(setq p1 (getpoint (list 100 200)))
```

```
(setq p2 (getpoint p1 "Input second point: "))
```




1、数学函数

2、交互函数

语法格式	说明 (角点和距离)
<code>(getangle [base] [prompt])</code>	返回以弧度为单位的角度值。 可以输入角度、点来响应。
<code>(getcorner base [prompt])</code>	以基点为一角点，矩形提示。 返回选定的另一角点坐标。
<code>(getdist [base] [prompt])</code>	返回距离（实数）值。无base输入数或两点，有输数值或一点。





1、数学函数

2、交互函数

举例（必须单句执行，等候输入）：

```
(getangle "Get a Angle: ")
```

输入90，得到1.5708；

输入45，得到0.785398；

输入一个数来指定一个角度，函数返回角的弧度值。

```
(getangle '(0 0) "Get Angle From a Point: ")
```

等候输入一个点，它与基点的连线与X轴的夹角即为返回的角度值，例如：输入0，30回车后为1.5708

```
(setq dist(getdist))
```

输入一个距离值，例如：输入300返回300

```
(setq dist(getdist '(0 0)))
```

等候输入一个点，它与基点（0 0）间的距离即为要输入的距离，例如：输入300，400回车后500



1、 数学函数

2、 交互函数

3、 表处理

•LISP代表表处理（List processing）。在LISP中，表是建立数据和程序结构的主要工具。由于LISP的数据和程序都是表，表处理函数显得非常重要。

语法格式	说明(构表函数)
(list <express1>...)	将各个表达式的返回值组成表。
(cons <element> <list>)	将element 作为第一个元素加入到list中，返回所构成的新表。
(append <list1> <list2>...)	将任意多个表合成一个表，新表的元素是按原来各表的元素顺序依次拷贝的。





1、数学函数

2、交互函数

3、表处理

举例（可整体执行）：

```
(list 1 2 3)
```

```
(cons 1 (list 2 3))
```

```
(append (list 1 2) (list 2 3))
```

```
(1 2 3)
```

```
(1 2 3)
```

```
(1 2 2 3)
```

说明：

- 1) 注意各函数的差别；
- 2) 特别注意返回值；
- 3) 为使Autolisp不对表求值，
要在表的左小括号前加单引号“`'`”，
或表内最左边加list。



1、数学函数

2、交互函数

3、表处理

语法格式	说明(读取函数)
(car <list>)	返回表list的第一个元素。
(cdr <list>)	返回除去list中的第一个元素之外的剩余元素所构成的新表。
(last <list>)	返回表的最后一个元素。
(nth <n> <list>)	返回表list中的第n个元素。表中元素的顺序从左到右依次为第0, 1, 2, ..., N-1。N为表的长度。





1、 数学函数

2、 交互函数

3、 表处理

举例 （可整体执行）：

```
(car (list 1 2 3))
```

```
(cdr (list 1 2 3))
```

```
(last (list 1 2 3))
```

```
(nth 1 (list 1 2 3))
```

1

(2 3)

3

2

说明： 1） 注意各函数的差别；
2） 特别注意返回值；





1、数学函数

2、交互函数

3、表处理

4、逻辑函数

语法格式	说明(逻辑运算)
(AND express1 ...)	从左到右对每个express求值，当有值为 nil时，停止求值，并返回 nil，否则返回T。
(OR express1 ...)	左到右对每个express求值，当有值为非nil时，停止求值，并返回 T，否则返回nil。
(NOT express)	若express为nil，返回T。



1、数学函数

2、交互函数

3、表处理

4、逻辑函数

5、关系函数

语法格式	说明 (数值比较)
$(< \text{exp1} \text{ exp2} \dots \text{expn})$	仅当 $\text{exp1} < \text{exp2} < \dots < \text{expn}$ 时，函数返回T，否则返回NIL。
$(> \text{exp1} \text{ exp2} \dots \text{expn})$	仅当 $\text{exp1} > \text{exp2} > \dots > \text{expn}$ 时，函数返回T，否则返回NIL。
$(<= \text{exp1} \text{ exp2} \dots \text{expn})$	小于等于函数。
$(>= \text{exp1} \text{ exp2} \dots \text{expn})$	大于等于函数。



1、数学函数

2、交互函数

3、表处理

4、逻辑函数

5、关系函数

语法格式	说明 (关系比较)
$(= \text{exp1} \text{exp2} \dots \text{expn})$	仅当 $\text{exp1}=\text{exp2}=\dots=\text{expn}$ 时，函数返回T，否则返回NIL。
$(\neq \text{exp1} \text{exp2} \dots \text{expn})$	仅当 $\text{exp1}\neq\text{exp2}\neq\dots\neq\text{expn}$ 时，函数返回T，否则返回NIL。





1、 数学函数

2、 交互函数

3、 表处理

4、 逻辑函数

5、 关系函数

举例（可整体执行）：

(`< 1 2 3`)

T

(`> 1 2 3`)

nil

(`< 1 1 3`)

nil

(`<= 1 1 3`)

T

(`= 1 1 3`)

nil

(`/= 1 1 3`)

nil

说明：1）注意各函数的差别；
2）特别注意返回值



6.4 程序设计简介

- AutoLISP程序的主要用途就是用于对AutoCAD进行开发。
- 开发办法就是用LISP编写用户自己的函数，通过自己的函数来实现特有的计算、绘图功能。





1、自定义函数

命令关键字

函数名

格式:

```
(defun < name> ( [arg] / [var] )  
  express1  
  .....  
)
```

在函数的变量表之后是函数体，它由一个以上的表达式express组成。

形参(argument)
是函数从外部取得变量，在函数被调用时形参从对应的实参中获取值。

局部变量(variable)不能直接从外部取得数据，它只能在函数内部被赋值和使用。



1、自定义函数

希望所定义的函数成为AutoCAD的一个命令时，

命令关键字

格式：

```
(defun c:< name> (  
  express1  
  .....  
)
```

函数名

空变元表

在函数的变量表之后是函数体，它由一个以上的表达式express组成。

定义AutoCAD命令与定义函数基本相同，不同之处是：

① 命令名前加“C:”。

② 没有变元，但可以有局部变量说明。

此外要注意所定义的AutoCAD命令不应与现有的AutoCAD命令同名。



1、 自定义函数

2、 IF 分支函数

IF格式: (IF test1 express1 [express2])

控制变量

否则执行
express2。
只能一个语句

若test1不为
nil, 执行
express1语句。
只能一个语句





1、 自定义函数

2、 IF 分支函数

3、 PROGN函数

语法格式：(PROGN express1 express2...)

函数关键字

要顺序执行的语句

功能作用：

扩展IF语句，使之可执行多条语句。



1、 自定义函数

2、 IF 分支函数

3、 PROGN函数

4、 COND多支函数

语法格式:

```
(COND (test1 express11 express12... )  
      (test2 express21 express22... )  
      )
```

条件变量1

若test1不为nil, 执行
express11 及其后语句

若test1为nil, 检查test2
若test2不为nil, 执行
express21及其后的语句。

注意: 只执行一个条件





1、 自定义函数

2、 IF 分支函数

3、 PROGN函数

4、 COND多支函数

5、 Repeat循环

6、 while 循环

语法格式:

(repeat <times> <express> ...)

要循环的次数

要循环执行的语句

While循环语法格式:

```
(while < test >  
    < express1 >  
    < express2 >  
    ...  
)
```

执行方式:

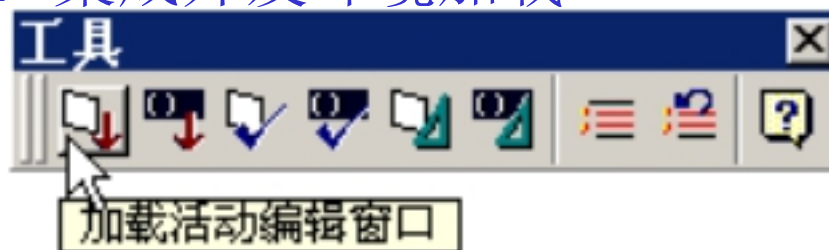
循环变量test不为nil时, 依次执行循环体中的语句; 否则结束循环



■ 加载、运行AutoLISP 源程序（两种方式）

① 通过Visual LISP 集成开发环境加载

I. 工具栏加载



II. 菜单加载

A. [文件] → [加载文件...]

B. [工具] → [加载编辑器中的文字]

III. 运行

A. “defun c:<函数名>”在控制台窗口中，在\$提示符后输入： (c:函数名) 后回车

B. “defun <函数名>”在命令窗口中的“在控制台窗口中，在\$提示符后输入： (函数名)，如有参数在()内加上相应参数值，函数名与参数值之间空一格，回车



- 在 AutoCAD 环境中加载、运行AutoLISP程序
 - I. 命令行加载
 - A. **APLOAD**
 - II. 菜单加载
 - A. **[工具] → [AutoLISP] → [加载...]**
 - III. 运行
 - ✓ “defun c:<函数名>”在命令窗口中的“命令：”提示行输入：**函数名**，回车
 - ✓ “defun <函数名>”在命令窗口中的“命令：”提示行输入：**(函数名)**，如有参数在（）内加上相应参数值，函数名与参数值之间空一格，回车





1、 自定义函数

2、 IF 分支函数

3、 PROGN函数

4、 COND多支函数

5、 Repeat循环

6、 while 循环

7、 应用举例

自定义函数举例：

```
(defun comp (a b / c)
  (setq c (getint "\n Input a Integer "))
  (if (<= (+ a b) c)
    "(a+b) LESS OR EQUAL c"
    "(a+b) MORE THAN c")
)
```

说明：

- 1) 在记事本中键入后保存为*.lsp的文件；
- 2) 用appload载入；
- 3) 执行

命令行键入：(comp 1 2)， 回车后执行
Input a Integer 10

"(a+b) LESS OR EQUAL c"



1、 自定义函数

2、 IF 分支函数

3、 PROGN函数

4、 COND多支函数

5、 Repeat循环

6、 while 循环

7、 应用举例

定义**AutoCAD**的新命令：

又如：通过三点绘圆弧和三角形

```
(defun c:Mydraw()  
  (setq p1 (getpoint "\n输入第一点:"))  
  (setq p2 (getpoint "\n输入第二点:"))  
  (setq p3 (getpoint "\n输入第三点:"))  
  (command "arc" p1 p2 p3) ;通过3点,  
  绘圆弧  
  (command "line" p1 p2 p3 "c") ;通过  
  3点绘三角形, “c”表示三角形结束  
)
```





1、 自定义函数

2、 IF 分支函数

3、 PROGN函数

4、 COND多支函数

5、 Repeat循环

6、 while 循环

7、 应用举例

COND多分支程序举例：

```
(defun c:comp ()  
  (setq c (getint "\n Input a Integer "))  
  (cond ((<= c 60) "No Pass")  
        ((<= c 85) "Pass")  
        ((> c 85) "Excellent")  
  )  
)
```

说明： (defun c:××× () <表达式>...) 作用是用来增加AutoCAD命令。

1) 在记事本中键入后保存为 *.lsp的文件；

2) 用appload载入；

3) 执行：

命令行键入： **comp** 回车后执行

Input a Integer 10

"No Pass"



1、 自定义函数

2、 IF 分支函数

3、 PROGN函数

4、 COND多支函数

5、 Repeat循环

6、 while 循环

7、 应用举例

Repeat循环程序举例：

```
(defun c:char()  
  (setq i 0 )  
  (repeat 26  
    (write-char (+ 65 i))  
    (setq i (1+ i))  
  )  
)
```

说明：

(write-char <数>) 将一个字符写到屏幕，其中
<数>是要写字符的ASC II 码。

- 1) 在记事本中键入后保存为 *.lsp的文件；
- 2) 用upload载入；
- 3) 执行：

命令: char

ABCDEFGHIJKLMNOPQRSTUVWXYZ26



1、 自定义函数

2、 IF 分支函数

3、 PROGN函数

4、 COND多支函数

5、 Repeat循环

6、 while 循环

7、 应用举例

命令: **Score**

Input a Name: xiaoming

Input a Score: 95

Pass

1

IF分支和PROGN函数程序举例:

```
(defun c:Score ()  
  (setq name (getstring T "\n Input a Name: "))  
  (setq score (getint "\n Input a Score: "))  
  (if (= No-Pass nil) (setq No-Pass 0))  
  (if (= Pass nil) (setq Pass 0)) ; 计数开始赋值为0  
  (if (< score 60)  
    (progn  
      (setq No-Pass (+ No-Pass 1)) ; 计数开始  
      (princ "No Pass\n") ; 输出no-pass, 换行  
      (princ No-Pass) ; 输出no-pass 变量值  
      (princ) ;屏蔽返回结果, 使该函数无返回值  
    )  
    (progn  
      (setq Pass (+ Pass 1))  
      (princ "Pass\n")  
      (princ Pass)  
      (princ)  
    )  
  )  
)
```




6.5 应用举例

1、几个例子

1) 正弦曲线

程序原理：

画正弦曲线

变量a: 幅值

fai: 初相位角

dfai: 相位角增量

$$y=100+a*\sin(wx+fai)$$

程序举例：

```
(defun c:sinx()
  (setq a (getreal "\n Input the Amplitude: "))
  (setq fai (getreal "\n Input the Phase: "))
  (setq fai (* (/ fai 180.0) pi))
  ;把相位角变成弧度
  (setq p0 (list 0 (+ 100 (* a (sin fai)))))
  ;计算起始点坐标
  (command "line" "0,100" "400,100" "")
  (command "line" "0,0" "0,200" "")
  ;画坐标轴线
  (setq x 1) ;设循环变量初值
  (setq dfai (/ pi 20)) ;计算相位角增量
  (while (<= x 40)
    (setq fai (+ fai dfai))
    (setq y1 (+ (* a (sin fai)) 100))
    (setq x1 (* x 10))
    (setq p1 (list x1 y1))
    (command "line" p0 p1 "")
    (setq p0 p1)
    (setq x (+ x 1))
  )
)
```



1、几个例子

1) 正弦曲线

对数曲线

```
程序:
(defun c:lnx()
  (setq value (getreal "\n 请输入X值: ")) ;输入绘图区间
  (if(<= value 0)
    (progn
      (princ "\n错误: X值不能小于等于0! ")
      (princ)
    ) ;第一progn结束
    (progn
      (setq delta (/ value 20)) ;把绘图区间分为20份
      (setq x delta)
      (setq n 0)
      (setq p0 (list delta (log delta))) ;计算起始点坐标
      (while (<= n 19)
        (setq x (+ x delta) y (log x))
        (setq p1 (list x y))
        (command "line" p0 p1 "")
        (setq p0 p1)
        (setq n (+ n 1))
      ) ;while循环结束
      (command "line" (list 0 (min 0 (log delta)))
        (list 0 (max 0 (log value))) "")
      (command "line" "0,0" (list value 0) "") ;画坐标轴线
      (command "zoom" "w"
        (list 0 (min (log value) (log delta)))
        (list value (max (log value) (log delta)))
      )
    ) ;第二progn结束
  ) ;if(= value 0)结束
) ;画对数曲线的函数结束
```



1、几个例子

1) 正弦曲线

2) 计算Ln (1+x)

程序原理:

用泰勒级数来计算

$$\ln(1+x) = \sum (-1)^{n+1} \times x^n / n$$

(n=1,2,3,.....)

$x \in (-1, 1]$

命令: (ln 3)

x must between -1 and 1 !

命令: (ln 0.5)

0.405468

程序:

```
(defun ln(x / n y temp result)
```

```
(if (And (> x -1) (<= x 1))
```

```
(progn
```

```
(setq n 1; 设n=1表示计算的阶次
```

```
y 1; 设y=1, 保证能循环计算
```

```
temp -1.0; 设临时变量temp=-1.0
```

```
result 0.0; 设计算结果 result=0.0
```

```
) ;progn系列中的第一句结束
```

```
(while (> (abs y) 1e-5);如果误差值大于1e-5则循环
```

```
(setq temp (* temp x -1) ; temp=temp*x*(-1)
```

```
y (/ temp n) ; y=temp/n
```

```
n (+ 1 n) ; n=n+1为下一次循环做准备
```

```
result (+ result y) ; result=result+y
```

```
)
```

```
) ;progn系列中的第二句—while循环结束
```

```
(princ result) ;progn系列中的第三个语句
```

```
) ;progn循环结束
```

```
(prompt " x must between -1 and 1 !\n" )
```

```
; if不成立执行, 在屏幕上显示提示
```

```
) ; if循环结束
```

```
(princ) ;屏蔽返回结果, 使该函数无返回值
```

```
) ;defun结束
```



1) 正弦曲线

2) 计算 $\ln(1+x)$

3) 标注粗糙度



```
(defun c:ccd1()
```

```
(setq P0 (getpoint "请输入插入点的坐标: "))
```

```
(setq alf (getangle "请输入旋转角度: "))
```

```
(setq text0 (getstring "请输入粗糙度的Ra值: "))
```

下面计算各绘图参数

```
(setq h0 (getvar "dimtxt"))
```

;读取系统变量dimtxt, 文字高度

```
(setq df (getvar "dimscale"))
```

;读取系统变量dimscale, 比例因子

```
(setq a0 (* 1.6 h0 df))
```

```
(setq a1 (* 1.4 h0 df))
```

```
(setq rad (/ pi 180))
```

pi是AutoLISP内部定义的常数, 3.1415926

```
(setq alf1 (/ alf rad)) ;alf的度数alf1
```

```
(setq bt1 (* (+ alf1 60) rad))
```

```
(setq bt2 (* (+ alf1 120) rad))
```

```
(setq bt4 (* (+ alf1 90) rad))
```

```
(setq x1 (+ (car p0) (* a0 (cos bt1))))
```

```
(setq y1 (+ (cadr p0) (* a0 (sin bt1))))
```

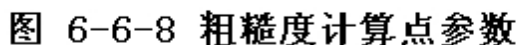
```
(setq p1 (list x1 y1))
```



1) 正弦曲线

2) 计算 $\ln(1+x)$

3) 标注粗糙度



```
(setq x2 (+ (car p0) (* a0 (cos bt2))))
(setq y2 (+ (cadr p0) (* a0 (sin bt2))))
(setq p2 (list x2 y2))
(setq x3 (+ (car p0) (* 2.0 a0 (cos bt1))))
(setq y3 (+ (cadr p0) (* 2.0 a0 (sin bt1))))
(setq p3 (list x3 y3))
(setq x4 (+ (car p0) (* 1.5 a1 (cos bt4))))
(setq y4 (+ (cadr p0) (* 1.5 a1 (sin bt4))))
(setq p4 (list x4 y4))
```

下面开始绘图

```
(command "Line" p1 p2 p0 p3 "")
(if (or (and (<= alf1 -90) (>= alf1 - 240))
      (and (<= alf1 270) (>= alf1 120)))
```

);end or

```
(setq alf1 (+ alf1 180))
```

);endif

```
(command "text" "m" p4 h0 alfl text0)
```

);defun结束



1、几个例子

1) 正弦曲线

2) 计算 $\ln(1+x)$

3) 标注粗糙度

4) 绘制三角形

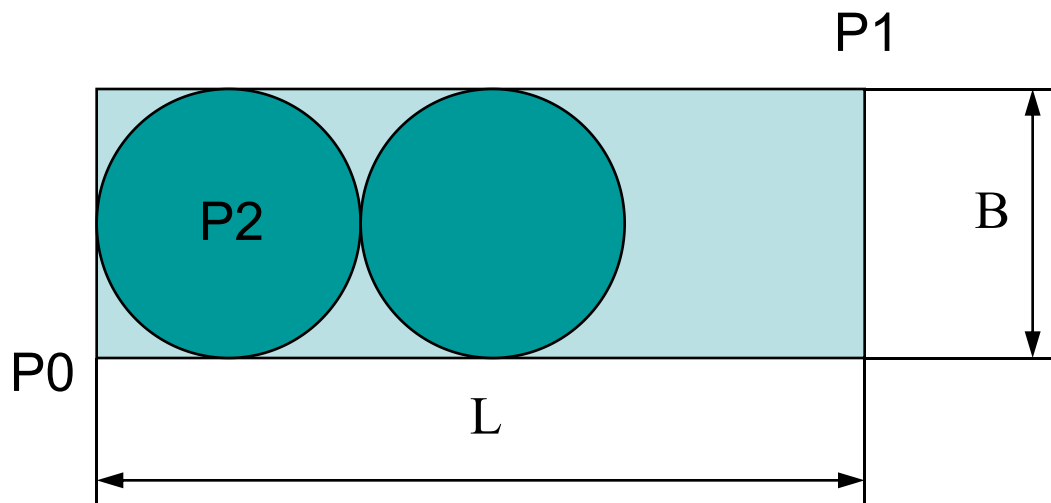
```
(defun c:lw1()  
  (setq x1 (getpoint "\n请输入第一点:"))  
  (setq x2 (getpoint "\n请输入第二点:"))  
  (setq x3 (getpoint "\n请输入第三点:"))  
  (command "line" x1 x2 "")  
  (command "line" x2 x3 "")  
  (command "line" x3 x1 "")  
)
```





2、综合举例

- 1) 已知一板长 L ，宽为 B ，若在其上截取直径为 B 的圆形板料，画出排料图。



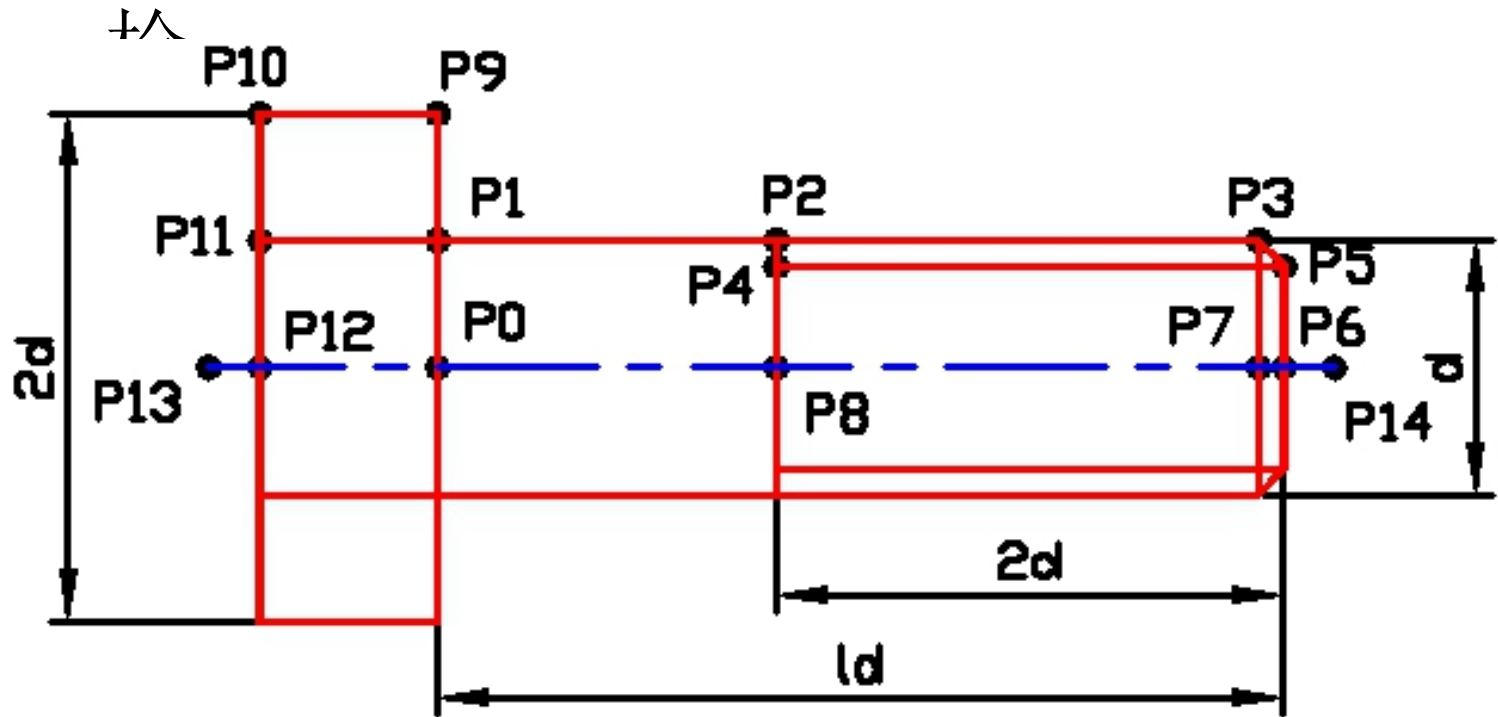


```
(defun c:div () ;自定义一个排料命令div
  (setq p0 (getpoint "\n输入基点: ")) ;输入P0点坐标
  (setq L (getreal "L= ")) ;输入板料长度L
  (setq B (getreal "B= ")) ;输入板料宽度B
  (setq R (* B 0.5)) ;计算圆的半径
  (setq p1 (polar (polar p0 0 L) (/ PI 2) B)) ;计算P1点
  (command "rectangle" p0 p1) ;画矩形
  (setq num (fix (/ L B))) ;取整
  (setq a 0) ;初始化循环变量
  (setq p2 (polar (polar p0 (/ PI 2) R) 0 R)) ;计算圆心坐标
  (while (< a num)
    (command "circle" p2 R)
    (setq p2 (polar p2 0 B))
    (setq a (1+ a)) ;(setq a (+ 1 a))
  )
)
```





2) 下面是一个绘螺栓的AutoLISP程序。
执行该程序，AutoCAD依次提示“确定基点”、“输入螺栓直径”和“输入螺栓长度”，之后即可绘出对应螺





(defun c:lwz() ;自定义一个绘螺栓的命令

(setq p0 (getpoint "确定基点:")) ;提示用户输入定义螺栓位置的基点

(setq d (getreal "\n输入螺栓直径:")) ;提示用户输入螺栓直径

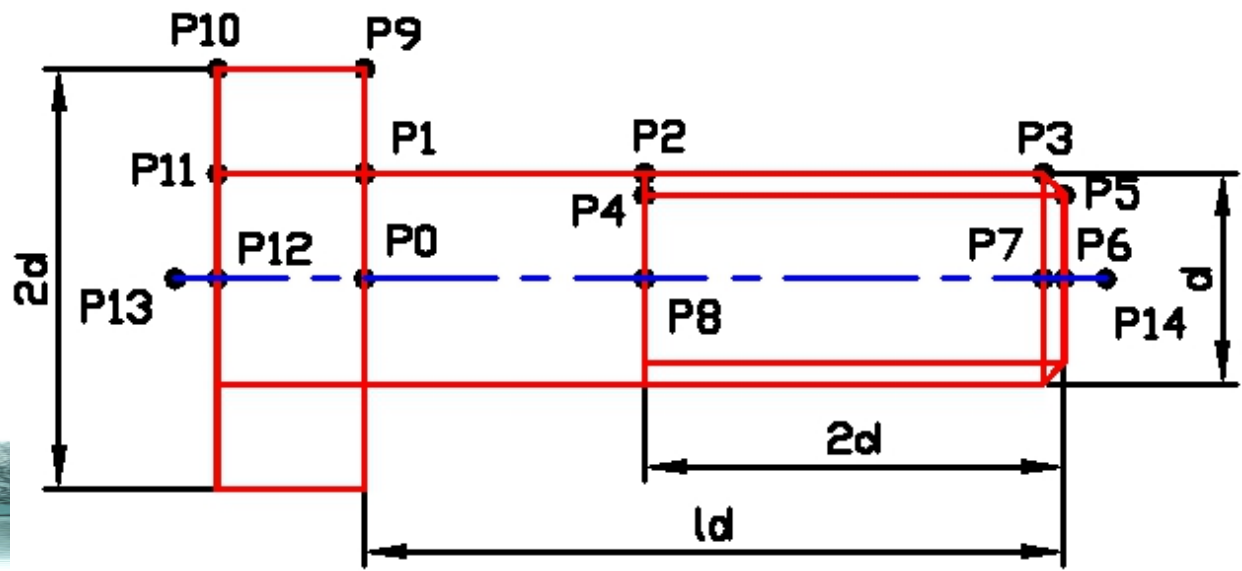
(setq ld (getreal "\n输入螺栓长度:")) ;提示用户输入螺栓长度

(if (> ld (* 2 d)) ;如果螺栓长度大于两倍直径，将按实际输入值绘制

(princ "OK")

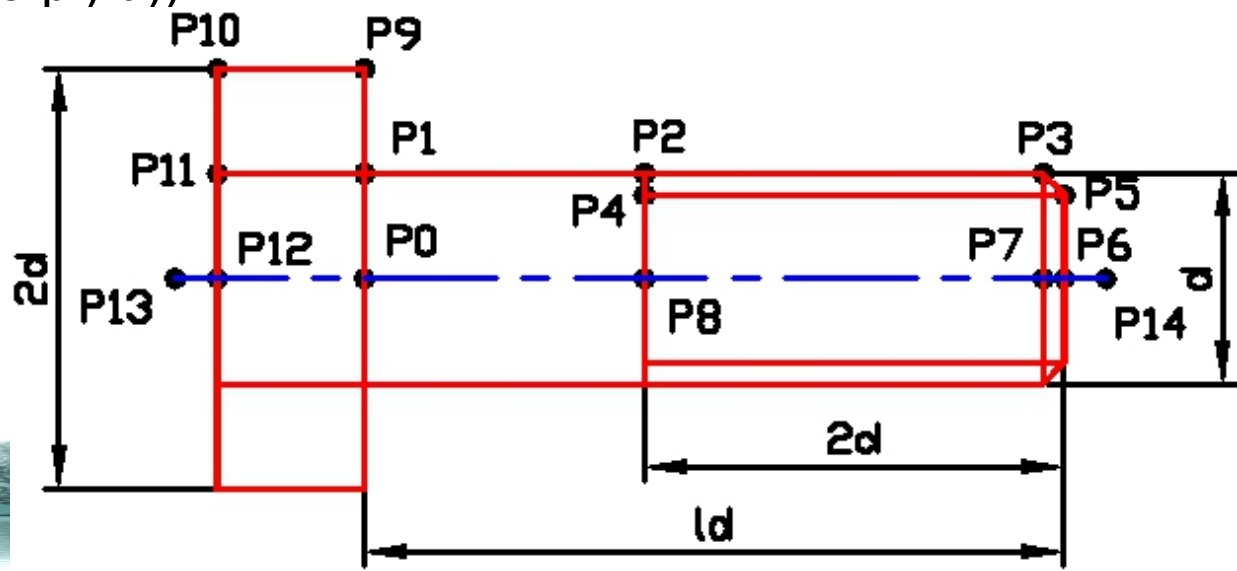
(setq ld (* 2 d)) ;如果螺栓长度小于两倍直径,让长度等于两倍的直径

)



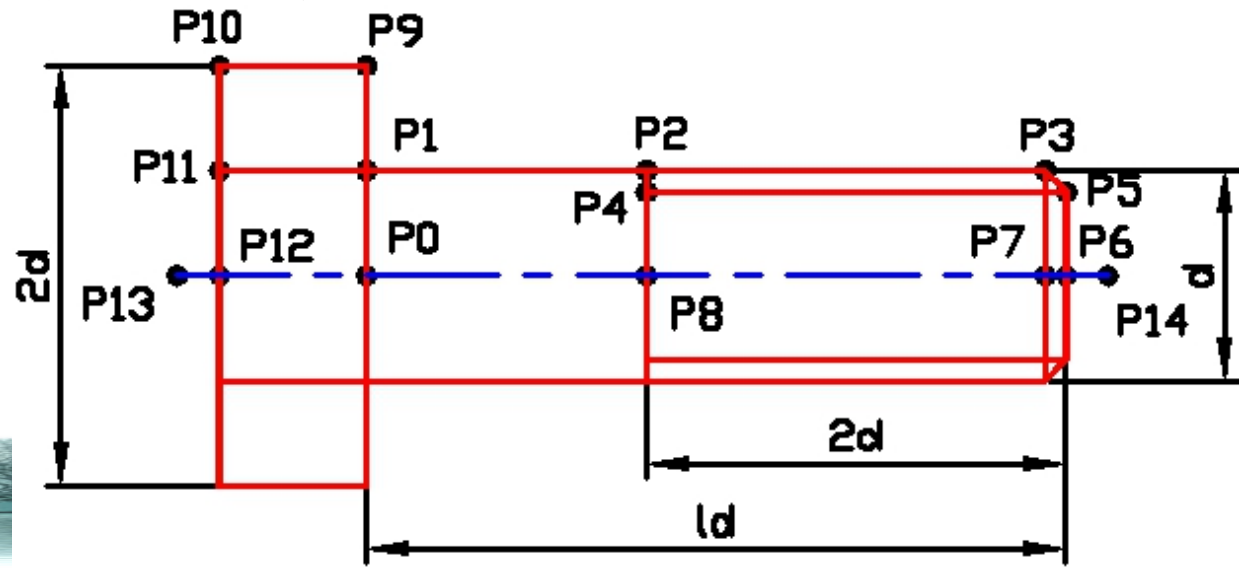


(setq p1 (polar p0 (* 0.5 pi) (* 0.5 d))) ;计算p1至p14点坐标.
(setq p2 (polar p1 0 (- ld (* 2 d)))) ;(polar <点> <角度> <距离>)
(setq p3 (polar p1 0 (- ld (* 0.1 d)))) ;返回一个点的坐标, 该点距<点>的
距离为<距离>,
(setq p4 (polar p2 (* -0.5 pi) (* 0.1 d))) ;与X轴正方向的夹角为<角度>
(setq p5 (polar p4 0 (* 2 d)))
(setq p6 (polar p0 0 ld))
(setq p7 (polar p0 0 (- ld (* 0.1 d))))
(setq p8 (polar p0 0 (- ld (* 2 d))))
(setq p9 (polar p0 (* 0.5 pi) d))



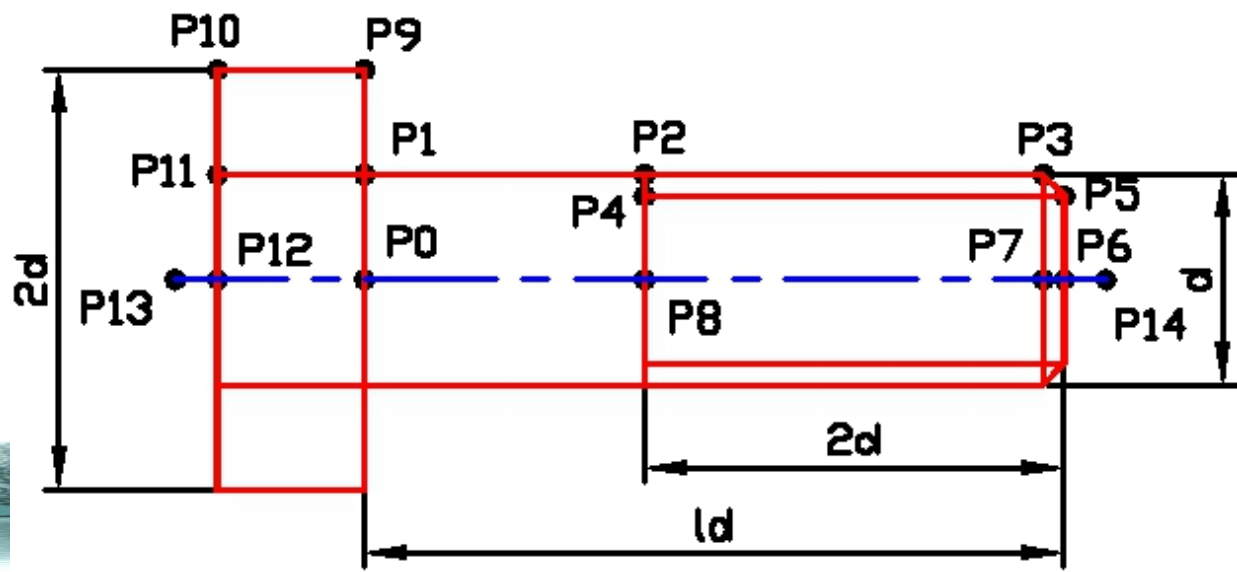


```
(setq p10 (polar p9 pi (* 0.7 d)))  
(setq p11 (polar p1 pi (* 0.7 d)))  
(setq p12 (polar p0 pi (* 0.7 d)))  
(setq p13 (polar p0 pi (* 0.9 d)))  
(setq p14 (polar p0 0 (+ ld (* 0.2 d))))  
(setq ss (getvar "clayer")) ;取出当前层信息  
(command "layer" "m" "center"  
          "l" "center" "center" "");建立中心线层，并令其为当前层  
(command "line" p13 p14 "");绘中心线  
(setvar "clayer" ss) ;切换到连续线层
```





(command "pline" p0 "w" 0.6 0.6 p9 p10 p12 "");绘螺栓上半部分轮廓线
(command "pline" p11 p3 p5 p6 "")
(command "pline" p2 p8 "" "pline" p3 p7 "")
(command "line" p4 p5 "")
(command "mirror" "c" p10 p6 "" p0 p6 "n") ;镜像螺栓上半部分轮廓线
;mirror命令, 选择对象: c, 指定第一个角点: 指定对角点:
;选择对象: 指定镜像线的第一点: 指定镜像线的第二点:
;要删除源对象吗? [是(Y)/否(N)] <N>: n
(command "ltscale" 5) ;改变线型比例
(princ) ;打印空行
);程序结束





设置图层、颜色、线型和线宽

1. 创建一个当前图层:

➤ 假定图层的名字是“zhongxin”、颜色为红色、线型为center、线宽为0.2，通过command函数创建一个当前图层表达式如下：

```
(command "layer" "Make" "zhongxin" "Color" 1 "zhongxin" "Ltype" "Center" "zhongxin" "LWeight" 0.2 "zhongxin" "")
```

➤ 因为在命令行操作时，layer命令需要空回车响应 “[?/Make/Set/New/ON/OFF/Color/Ltype/LWeight/Plot/Freeze/Thaw/LOck/Unlock/state]”提示才能结束该命令，所以在右括号前增加一对引号（注意，引号内没有空格）。

➤ 因为Make、Color、Ltype、LWeight等选项可以简写为M、C、L、LW，所以上式可改为：

```
(command "layer" "M" "zhongxin" "C" 1 "zhongxin" "L" "Center" "zhongxin" "LW" 0.2 "zhongxin" "")
```





➤ 又因为当前图层的名字是**Color**、**Ltype**等选项默认的图层名，所以上式可改为：

```
(command "layer" "M" "zhongxin" "C" 1 "" "L" "Center" "" "LW" 0.2 "" "")
```

➤ 如果当前图层的颜色、线型、线宽等为默认的选择，表达式如下：

```
(command "layer" "M" "cuxian" "")
```

图层**cuxian**是当前图层、颜色号为7（白/黑）、线型为**continuous**、线宽为当前图形对象的默认线宽。

➤ 如果某图层已经存在，只是将其改变为当前状态，其表达式如下：

```
(command "layer" "M" "cuxian" "")或者
```

```
(command "layer" "S" "cuxian" "")
```





2. 设置新图形对象的颜色

(1) 通过command函数设置新图形对象的颜色

(command "color" 3) 或者

(command "color" "green"); 设置新图形对象的颜色为绿色

(2) 通过setvar函数设置新图形对象的颜色

(setvar "cecolor" "2")或者

(setvar "cecolor" "yellow"); 设置新图形对象的颜色为黄色





3. 设置新图形对象的线型

(1) 通过command函数设置新图形对象的线型

(command "linetype" "s" "center" "") ; 设置新图形对象的线型为中心线

(2) 通过setvar函数设置新图形对象的线型

(setvar "celtype" "dashed"); 设置新图形对象的线型为虚线





4. 设置线型比例因子的大小

除了实线（continuous）之外，每种线型都是由不同长度的短划线、空白段或点组成的。在不同的显示比例下，这些短划线和空白段的视觉效果可能过大或过小。改变线型比例因子的大小并不改变整条线段的长度，只改变短划线和空白段的大小。例如，将线型的短划线和空白段缩小一半。

(1) 通过command函数设置线型比例因子的大小

```
(command "ltscale" 0.5)
```

(2) 通过setvar函数设置设置线型比例因子的大小

```
(setvar "ltscale" 0.5)
```



5. 设置新图形对象的线宽

(1) 通过command函数设置新图形对象的线宽

`(command "lweight" 0.5);` 设置新图形对象的线宽为0.5

(2) 通过setvar函数设置新图形对象的线宽

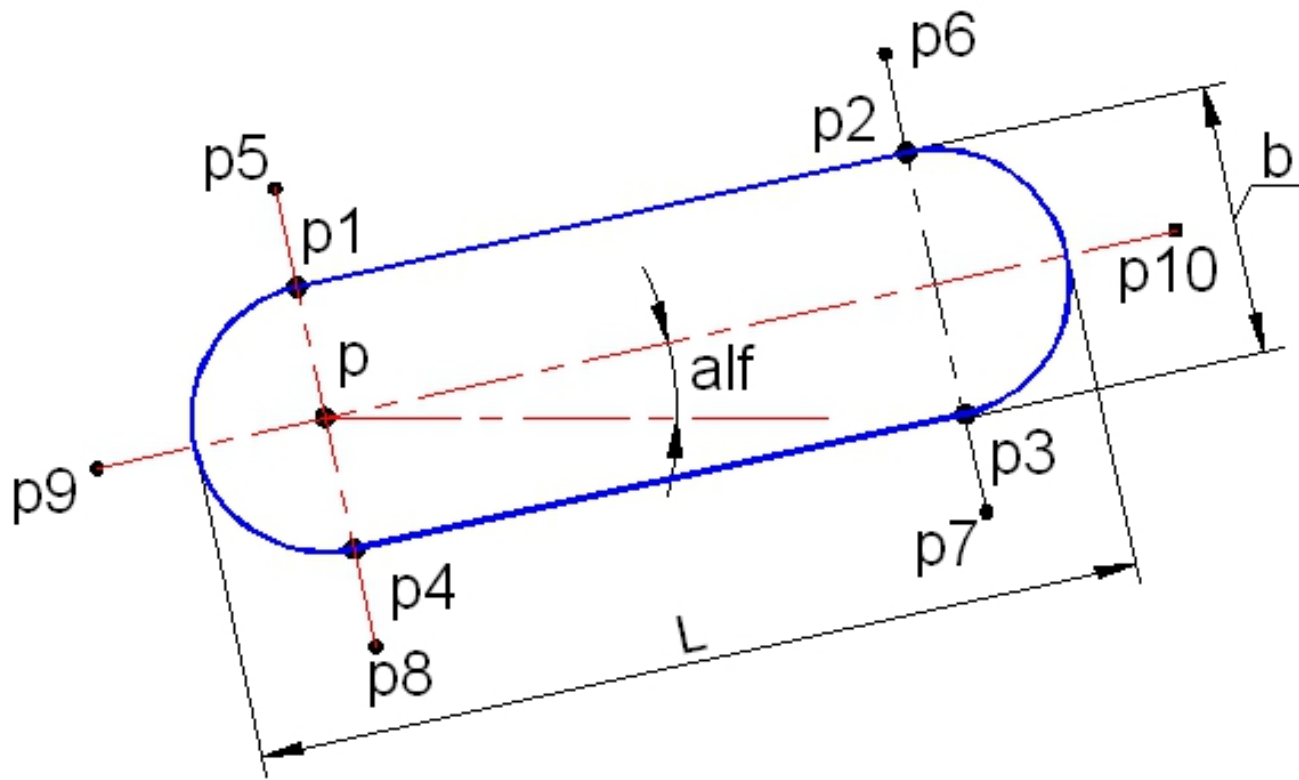
系统变量celweight记录着新图形对象的线宽，它的值是整型的，以1%为单位。例如设置新图形对象的线宽为0.5的表达式如下：

`(setvar "celweight" 50)`





【例】 定义绘制下图所示的平键外形轮廓线和中心线的命令，插入点 p 、键宽 b 、键长 L 、旋转角 alf 是交互输入的参数，要求粗实线所在图层的名字为`cuxian`、蓝色；中心线所在图层的名字为`zhongxin`，红色，线型为`center`（中心线）。





1. 定义绘制平键轮廓线命令的源程序

```
(defun c:pingjian(/ p b l alf r ll p1 p2 p3 p4 p5 p6  
p7 p8 p9 p10)
```

(setvar "osmode" 0) ; 关闭目标捕捉状态

(setvar "cmdecho" 0) ; 抑制普通命令的提示,但并不抑制
get×××函数的提示

(command"layer" "make" "cuxian" "c" 5 "cuxian" "")
; 生成粗实线所在的图层, 状态为当前层, 名字为cuxian, 颜色为蓝

(setq p(getpoint "输入平键的定位点: ")); 以交互方式
得到平键的定位点p

(setq b(getdist p "输入平键的宽度: ")); 以交互方式得
到平键的宽度b

(setq l(getdist p "输入平键的长度: ")); 以交互方式得
到平键的长度l



(setq alf (getangle p "输入平键的旋转角: ")); 以交互方式得到平键的旋转角alf

(setq r (* 0.5 b) 11(- 1 b))

(setq p1 (polar p (+ alf (* 0.5 pi)) r))

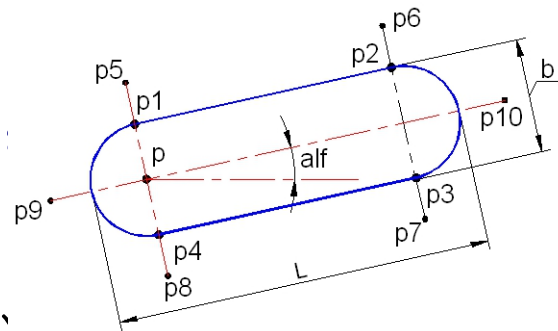
(setq p2 (polar p1 alf 11))

(setq p3 (polar p2 (+ alf (* 1.5 pi)) b))

(setq p4 (polar p1 (+ alf (* 1.5 pi)) b))

(command "pline" p1 "width" 0.6 0.6 p2 "arc" p3 "line" p4 "arc" "close"); 绘制平键的外形轮廓线

(command "layer" "m" "zhongxin" "c" 1 "" "1" "center" "" ""); 生成中心线所在的图层，状态为当前层，名字为zhongxinxian，颜色为红，线型为中心线





```
(setvar "ltscale" 5)
```

```
(setq p5 (polar p1 (+ alf (* 0.5 pi)) r))
```

```
(setq p6 (polar p2 (+ alf (* 0.5 pi)) r))
```

```
(setq p7 (polar p3 (+ alf (* 1.5 pi)) r))
```

```
(setq p8 (polar p4 (+ alf (* 1.5 pi)) r))
```

```
(setq p9 (polar p (+ alf pi) (+ r r)))
```

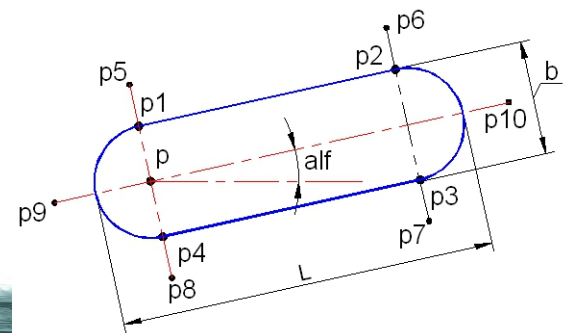
```
(setq p10 (polar p alf (+ (- 1 r) r)))
```

```
(command "line" p5 p8 ""); 绘制中心线
```

```
(command "line" p6 p7 "")
```

```
(command "line" p9 p10 "")
```

)





补充：尺寸标注

1. 长度型尺寸标注

长度型尺寸标注需要确定尺寸线是水平、垂直还是指定的方向，如果默认尺寸测量值即为尺寸文本，那么关键是确定其第1条尺寸界线起点的位置 $p1$ 、第2条尺寸界线起点的位置 $p2$ 和尺寸线的位置 $p3$ 。尺寸界线的起点通常与确定图形对象的点重合，因此可看作是已知的，于是剩余的工作就是确定位置尺寸线的位置 $p3$ 。

例如以下两个表达式，可得到下图所示的直线 $p1p2$ 。

```
(setq p1 '(10 10) p2 '(50 20) p3 '(30 25))
```

```
(command "line" p1 p2 "")
```





(1) 指定尺寸线为水平或垂直方向的长度型尺寸标注
(command "dimlinear" p1 p2 "h" p3)

该表达式的说明如下：

dimlinear: command函数调用标注长度型尺寸的命令。

p1、p2: 第1、2条尺寸界线起点的位置。

h: 尺寸线的方向。水平方向、垂直方向(v)还是指定其他方向(Rotated)。若选择其他方向，其后需增加角度数值。

p3: 确定尺寸线的位置点。可以是一个变量，如该例；可以是一个字符串"x, y"，串内的x、y确定了p3点在当前坐标系下的位置，如"30, 25"；也可以用相对于第2条尺寸界线的起点p2的位置来确定它，这样会方便得多。其格式如下：

"@ Δx , Δy "，@表示相对于， Δx 表示相对于p2点的X方向的增量， Δy 表示相对于p2点的Y方向的增量。



```
(command "dimlinear" p1 p2 "h" p3)
```

```
(command "dimlinear" p1 p2 "h" "30, 25")
```

```
(command "dimlinear" p1 p2 "h" "@-20, 5")
```

以上3个表达式用不同的形式确定了尺寸线的位置，见图a。

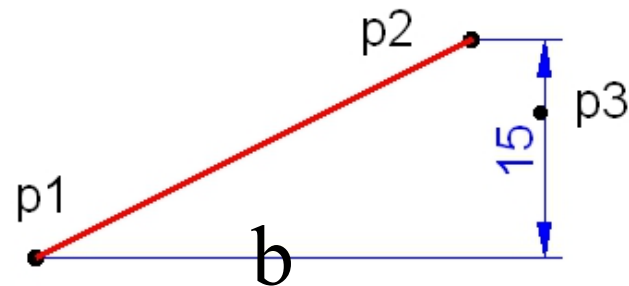
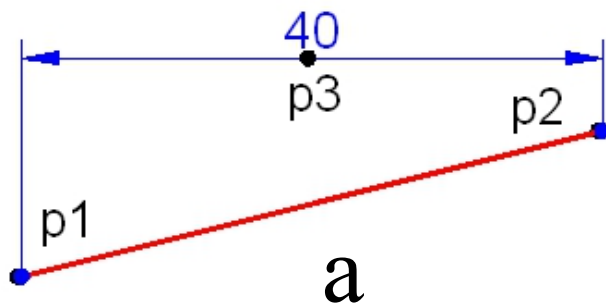
如果要标注尺寸线为垂直方向的尺寸，只需将以上表达式中的“h”改为“v”即可。例如：

```
(setq p1 '(10 10) p2 '(40 25) p3 '(45 20))
```

```
(command "line" p1 p2 "")
```

```
(command "dimlinear" p1 p2 "v" p3 )
```

结果见图b。





(2) 默认尺寸线方向的长度型尺寸标注

如果不指定尺寸线的方向，p3点与p1、p2点的相对位置确定了尺寸线的方向。若p3点的X坐标在p1、p2点的X坐标之外，而且p3点的Y坐标在p1、p2点的Y坐标之间，尺寸线为垂直方向；其他情况的尺寸线为水平方向。

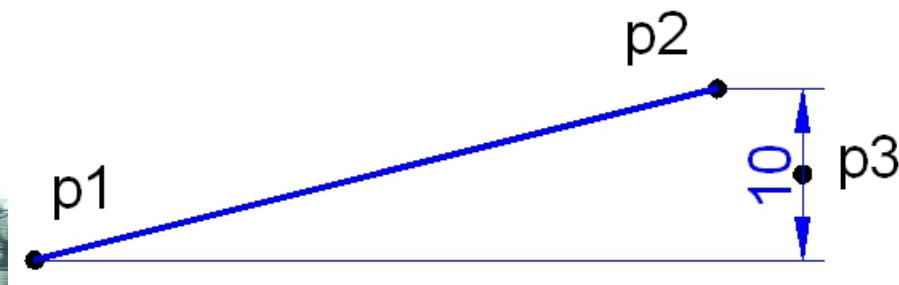
如果将去掉上述3个表达式中的“h”选项，其结果不变，因为p3点的X坐标在p1、p2点的X坐标之间。若改为以下表达式之一

```
(command "dimlinear" p1 p2 ' (55 15))
```

```
(command "dimlinear" p1 p2 "55,15")
```

```
(command "dimlinear" p1 p2 "@5,-5")
```

将得到该直线的尺寸线为垂直方向的标注，见下图。





2. 对齐型尺寸标注

对齐型尺寸标注的特点是，尺寸线的方向是确定的，它与被标注对象的两个端点或指定的两尺寸界线起点的连线平行。

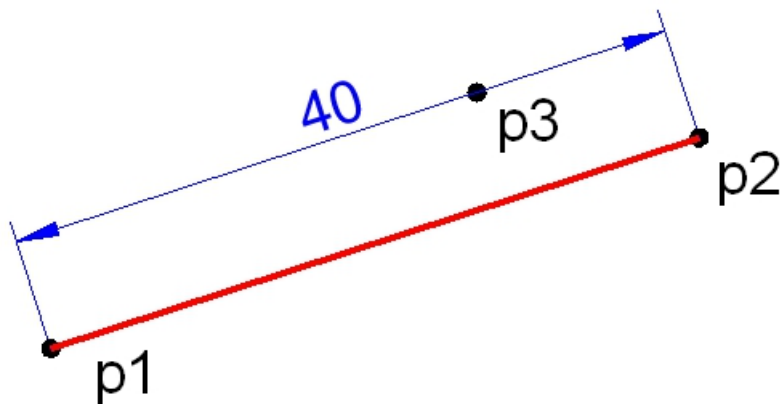
以下每个表达式都能生成下图所示的对齐型长度尺寸标注。

```
(command "dimaligned" p1 p2 p3)
```

```
(command "dimaligned" p1 p2 "35, 25")
```

```
(command "dimaligned" p1 p2 "@-13.04, 2.64")
```

表达式中各参数的含义与长度尺寸标注中各参数的含义相同。





3. 角度型尺寸标注

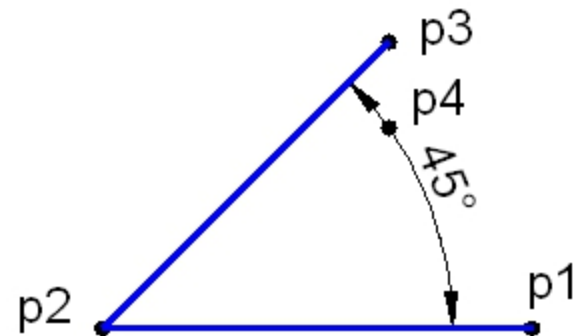
标注两条直线的夹角是最常见的角度型尺寸标注。需要首先选择被标注的两条直线，然后确定尺寸线的位置，如果默认尺寸测量值为尺寸文本，用""作为默认的选择即可。

以下每个表达式都能生成下图所示的角度型尺寸标注。

```
(command "dimangular" p1 p3 p4)
```

```
(command "dimangular" p1 p3 "40,14")
```

```
(command "dimangular" p1 p3 "@0,-6")
```



在上述表达式中，p1是第一条直线的选择点，p3是第二条直线的选择点，p4点确定了尺寸弧线的位置，字符串"40,14"是当前坐标系下的坐标，字符串"@0,-6"是相对于选择点p3的X方向的增量和Y方向的增量。由于默认尺寸的测量值，所以尺寸文本的最后一个字符为度的单位符号"°"。



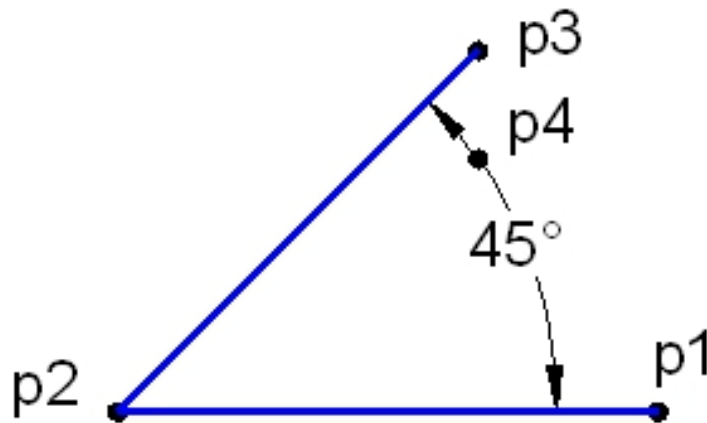
上图中尺寸文本“45°”的书写方向与尺寸弧线中点的切线方向相同，可能不符合某些单位的制图标准，改写为以下的表达式之一，可以得到下的结果。

```
(command "dimangular" p1 p3 "A" 0.01 p4)
```

```
(command "dimangular" p1 p3 "A" 0.01 "40,14")
```

```
(command "dimangular" p1 p3 "A" 0.01 "@0,-6")
```

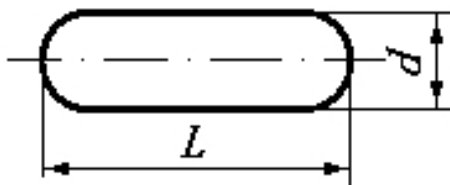
其中“A”是尺寸文本的旋转角度选项，其后是旋转角度的数值，由于AutoCAD不接受0°方向，因此取其近似值0.01。





本章作业

- 一. 用**LISP**语言编写一个实现如图所示平键的参数化绘图
的命令函数。（要求交互输入左端点作为插入点，长
度 L 和宽度 d ，可以不标注尺寸）



- 二. 定义一个新的AutoCAD命令drawdf，该命令根据用户
指定的圆心位置和半径绘圆以及圆的内接六边形。

